

JDBC

Java DataBase Connectivity

Contenido

JDBC	1
1.- Introducción.....	1
2.- Conectividad JDBC	2
3.- Establecimiento de conexiones.	3
4.- Codificación de Aplicaciones sobre bases de datos.	5
1º.- Descargar el controlador.	5
2º.- Registrar el controlador JDBC.	5
3º.- Ejecución de consultas sobre la base de datos.	7
4º.- Recuperación de información.	7
5º.- Actualización de información.	8
6º.- Añadir información.	9
7º.- Borrado de información.	9
8º.- Cierre de conexiones.	9
9º.- Excepciones en JDBC.	9
10º.- Sentencias preparadas para evitar SQLInjection.....	10
5.- Tipos SQL en Java	11
6.- Aproximación a SQL estándar.	13
Creación de Tablas	13
Recuperar Información	15
Almacenar Información	17
Borrar Información	18
Actualizar Información	18
Serialización	20
1.- Serialización	20
2.- Ejemplo de Lectura y Escritura de Objetos.....	20

JDBC

1.- Introducción.

Una Base de Datos es una serie de tablas que contienen información ordenada en alguna estructura que facilita el acceso a esas tablas, ordenarlas y seleccionar filas de las tablas según criterios específicos. Las tablas de las bases de datos generalmente tienen *índices* asociados a alguna de sus columnas, de forma que el acceso sea lo más rápido posible.

Las bases de datos responden a 3 modelos: Jerárquico, Relacional y Orientado a Objetos.

Un esquema de base de datos relacional define sus tablas, sus campos en cada tabla y las relaciones entre cada campo y cada tabla. El esquema es generalmente almacenado en un Diccionario de Datos.

El sistema gestor de bases de datos, en inglés conocido como: **Database Management System (DBMS)**, gestiona el modo en que los datos se almacenan, mantienen y recuperan.

En el caso de una base de datos relacional, el sistema gestor de base de datos se denomina: **Relational Database Management System (RDBMS)**.

JDBC (*Java DataBase Connectivity*) es un API de Java que permite al programador ejecutar instrucciones en lenguaje estándar de acceso a Bases de Datos, **SQL** (*Structured Query Language*, lenguaje estructurado de consultas), que es un lenguaje de muy alto nivel que permite crear, examinar, manipular y gestionar Bases de Datos relacionales. Para que una aplicación pueda hacer operaciones en una Base de Datos, ha de tener una conexión con ella, que se establece a través de un *driver*, que convierte el lenguaje de alto nivel en sentencias de Base de Datos.

Las tres acciones principales que realizará JDBC son:

- Establecer la conexión a una base de datos, ya sea remota o no.
- Enviar sentencias SQL a esa base de datos.
- Procesar los resultados obtenidos de la base de datos.

Existen multitud de bases de datos para PC, que son aquellas que usan la plataforma PC como cliente para acceder a un servidor. Estas bases de datos son *MySQL, Access, IBM DB/2, Microsoft SQL Server, Oracle, Sybase, SQLBase, Informix, XDB, Postgre, etc.* Todas éstas soportan varios dialectos similares de SQL, y todas parecen, a primera vista, intercambiables. La razón de que no lo sean es que cada una está diseñada con unas características de rendimiento distintas, con un interfaz de usuario y programación diferente. Aunque todas ellas soportan SQL y la programación es similar, cada base de datos tiene su propia forma de recibir las consultas SQL y su propio modo de devolver los resultados.

Aquí surge la idea de poder escribir código independiente de la bases de datos a la que queramos conectarnos.

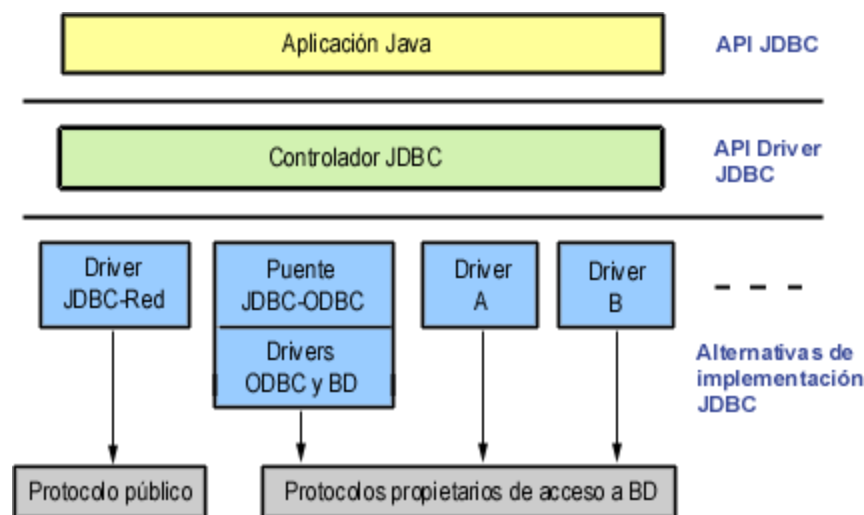
Con tal motivo Microsoft creó un nivel de estandarización llamado **ODBC** (*Open DataBase Conectivity*), aunque no es un driver rápido ni sofisticado lo que sí nos asegura es el perfecto funcionamiento en plataformas PC-Windows. Por tanto, muchos fabricantes de bases de datos soportan ODBC como un *interfaz alternativo* al suyo estándar.

2.- Conectividad JDBC

Cualquier sistema operativo -Windows en particular- no sabe nada acerca de las bases de datos, pero define el estándar ODBC consistente en un conjunto de primitivas que cualquier *driver* o fuente ODBC debe ser capaz de entender y manipular. Los programadores que a su vez deseen escribir programas para manejar bases de datos genéricas en Windows utilizan las llamadas ODBC.

Con respecto a Java, JDBC es una especificación de un conjunto de clases y métodos de operación que permiten a cualquier programa Java acceder a sistemas de bases de datos de forma homogénea. Lógicamente, al igual que ODBC, la aplicación de Java debe tener acceso a un driver JDBC adecuado. Este driver es el que implementa la funcionalidad de todas las clases de acceso a datos y proporciona la comunicación entre el API JDBC y la base de datos real.

La necesidad de JDBC, a pesar de la existencia de ODBC, viene dada porque ODBC es un interfaz escrito en lenguaje C, que al no ser un lenguaje portable, haría que las aplicaciones Java también perdiesen la portabilidad. Y además, ODBC tiene el inconveniente de que se ha de instalar manualmente en cada máquina; al contrario que los drivers JDBC, que al estar escritos en Java son automáticamente instalables, portables y seguros.



Toda la conectividad de bases de datos de Java se basa en sentencias SQL, por lo que se hace imprescindible un conocimiento adecuado de SQL para realizar cualquier clase de operación de bases de datos. Aunque, afortunadamente, casi todos los entornos de desarrollo Java ofrecen componentes visuales que proporcionan una funcionalidad suficientemente potente sin necesidad de que sea necesario utilizar SQL, aunque para usar directamente el JDK se haga imprescindible. La especificación JDBC requiere que cualquier driver JDBC sea compatible con al menos el nivel de ANSI SQL 92 (*ANSI SQL 92 Entry Level*).

El API JDBC viene distribuido en dos paquetes:

- `java.sql`, dentro de J2SE
- `javax.sql`, extensión dentro de J2EE

Un conector o driver es un conjunto de clases encargadas de implementar las interfaces del API y acceder a la base de datos.

Para poder conectarse a una base de datos y lanzar consultas, una aplicación necesita tener un conector adecuado. Un conector suele ser un fichero .jar que contiene una implementación de todas las interfaces del API JDBC.

Cuando se construye una aplicación de base de datos, **JDBC oculta los detalles específicos de cada base de datos**, de modo que le programador se ocupe solo de su aplicación.

El conector lo proporciona el fabricante de la base de datos o bien un tercero.

El código de nuestra aplicación no depende del driver, puesto que trabajamos contra los paquetes **java.sql** y **javax.sql**.

JDBC ofrece las clases e interfaces para:

- Establecer una conexión a una base de datos.
- Ejecutar una consulta.
- Procesar los resultados.

Ejemplo:

Código Java para establecer una conexión y ejecutar consulta:

// Establece la conexión

```
Connection con = DriverManager.getConnection ("jdbc:odbc:miBD", "miLogin", "miPassword");
```

// Ejecuta la consulta

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeQuery("SELECT nombre, edad FROM Jugadores");
```

// Procesa los resultados

```
while (rs.next()) {  
    String nombre = rs.getString("nombre");  
    int edad = rs.getInt("edad");  
}
```

3.- Establecimiento de conexiones.

JDBC define ocho interfaces para operaciones con bases de datos, de las que se derivan las clases correspondientes.

La clase que se encarga de cargar inicialmente todos los drivers JDBC disponibles es **DriverManager**. Una aplicación puede utilizar **DriverManager** para obtener un objeto de tipo conexión, **Connection**, con una base de datos. La conexión se especifica siguiendo una sintaxis basada en la especificación más amplia de los URL, de la forma

jdbc:subprotocolo//servidor:puerto/base de datos

Una vez que se tiene un objeto de tipo **Connection**, se pueden crear sentencias, **Statements**, ejecutables. Cada una de estas sentencias puede devolver cero o más resultados, que se devuelven como objetos de tipo **ResultSet**.

La tabla siguiente muestra la misma lista de clases e interfaces junto con una breve descripción.

Clase/Interface	Descripción
Driver	Permite conectarse a una base de datos: cada gestor de base de datos requiere un driver distinto
DriverManager	Permite gestionar todos los drivers instalados en el sistema
DriverPropertyInfo	Proporciona diversa información acerca de un driver
Connection	Representa una conexión con una base de datos. Una aplicación puede tener más de una conexión a más de una base de datos
DatabaseMetadata	Proporciona información acerca de una Base de Datos, como las tablas que contiene, etc.
Statement	Permite ejecutar sentencias SQL sin parámetros
PreparedStatement	Permite ejecutar sentencias SQL con parámetros de entrada
CallableStatement	Permite ejecutar sentencias SQL con parámetros de entrada y salida, típicamente procedimientos almacenados
ResultSet	Contiene las filas o registros obtenidos al ejecutar un SELECT
ResultSetMetadata	Permite obtener información sobre un ResultSet , como el número de columnas, sus nombres, etc.

Cuando se presenta con una URL específica, **DriverManager** itera sobre la colección de drivers registrados hasta que uno de ellos reconoce la URL especificada. Si no se encuentra ningún driver adecuado, se lanza una **SQLException**.

Código Java para establecer una conexión con MySQL. :

```
public static void main(String[] args) {  
    try {  
        // Cargar el driver de mysql, Java puede entenderse con MySQL.  
        Class.forName("com.mysql.jdbc.Driver");  
        /* Cadena de conexión para conectar con MySQL en localhost,  
        seleccionar la base de datos llamada "ejercicio"*/  
        String connectionUrl = "jdbc:mysql://localhost/ejercicio";  
        // Obtener la conexión con usuario y contraseña del servidor de MySQL: root y root  
        Connection con = DriverManager.getConnection(connectionUrl,"root","root");  
    } catch (SQLException e) {  
        System.out.println("SQL Exception: "+ e.toString());  
    } catch (ClassNotFoundException cE) {  
        System.out.println("Excepción: "+ cE.toString());  
    }  
}
```

Nota :

static Class	forName (String className) Returns the Class object associated with the class or interface with the given string name.
--------------	--

Es como una manera especial de hacer new, pero sin necesidad de importar el paquete donde está la clase.

4.- Codificación de Aplicaciones sobre bases de datos.

Pasos a seguir:

1º.- Descargar el controlador.

Descargarnos el conector o driver que necesitamos para trabajar con MySQL. Como verás, tan sólo consiste en descargar un archivo, descomprimirlo y desde NetBeans añadir el fichero **.jar** que constituye el conector que necesitamos.

El IDE Netbeans lo lleva incorporado en sus librerías. Se trataría simplemente de añadirla en la carpeta Libraries como más adelante se ejemplifica.

2º.- Registrar el controlador JDBC.

Al fin y al cabo ya lo hemos visto en el ejemplo de código que poníamos antes, pero incidimos de nuevo. Registrar el controlador que queremos utilizar es tan fácil como escribir una línea de código.

Hay que consultar la documentación del controlador que vamos a utilizar para conocer el nombre de la clase que hay que emplear. En el caso del controlador para MySQL es "**com.mysql.jdbc.Driver**", o sea, que se trata de la clase **Driver** que está en el paquete **com.mysql.jdbc** del conector que hemos descargado, y que no es más que una librería empaquetada en un fichero .jar.

La línea de código necesaria en este caso, en la aplicación Java que estemos construyendo es:

```
// Cargar el driver de mysql  
Class.forName("com.mysql.jdbc.Driver");
```

Una vez cargado el controlador, es posible hacer una conexión al SGBD.

Hay que asegurarse que si no utilizáramos NetBeans u otro IDE, para añadir el .jar como hemos visto habría que incluirlo en el CLASSPATH que emplea nuestra máquina virtual, o bien en el directorio ext del JRE de nuestra instalación del JDK.

La instalación es muy sencilla: **basta con copiar el fichero del controlador JDBC en el directorio ext de nuestra instalación del JDK**. A partir de ese momento ya podemos empezar a utilizarlo.

Quizás te ayude a recordar el lugar donde debes colocarlo el hecho de que el controlador que viene en el fichero .jar no es más que una **librería externa** al JDK, y que si la metemos dentro de nuestro **entorno de ejecución (JRE)** no habrá problemas con el CLASSPATH, ya que el entorno de ejecución estará incluido en el CLASSPATH.

Por tanto, dentro de la carpeta de instalación de nuestro JDK

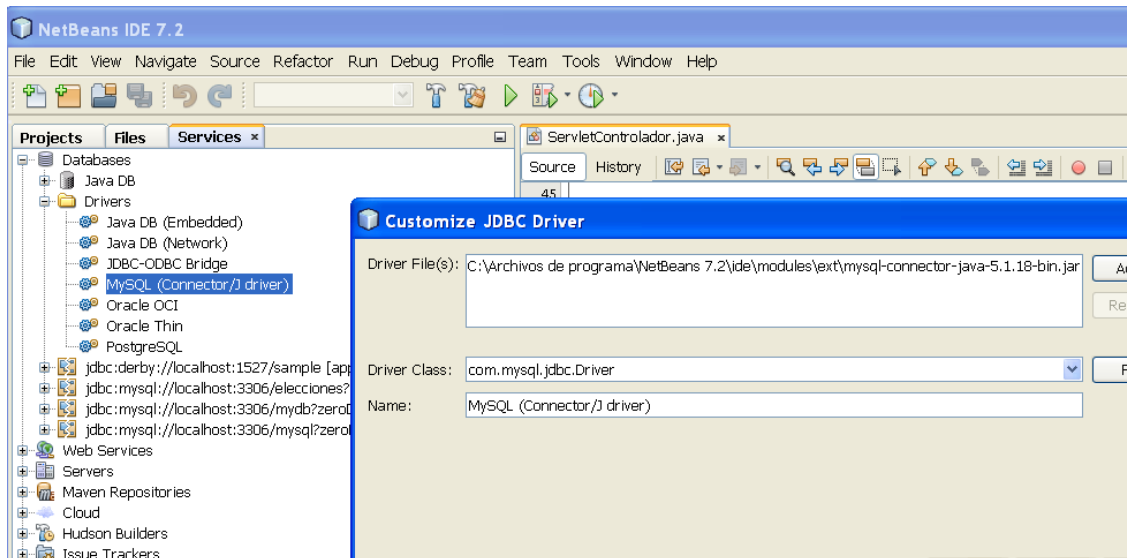
(C:\Archivos de programa\Java\jdk1.7.0, por ejemplo), debemos buscar la carpeta de instalación del JRE (C:\Archivos de programa\Java\jdk1.7.0\jre), y dentro la carpeta de librerías (C:\Archivos de programa\Java\jdk1.7.0\jre\lib), y dentro de ésta la carpeta de librerías externas (C:\Archivos de programa\Java\jdk1.5.0_04\jre\lib\ext).

Ya tenemos formada toda nuestra ruta, dentro de la que debemos colocar el conector o controlador JDBC para MySQL, que es el fichero **mysql-connector-java-5.1.22-bin** o el nombre que tenga en la última versión del mismo que te hayas descargado.

Una vez cargado el controlador, es posible hacer una conexión al SGBD.

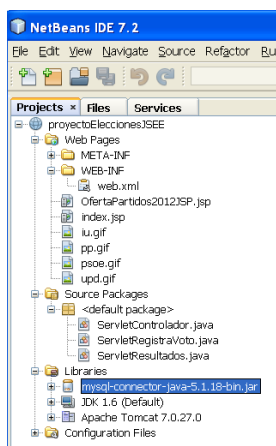
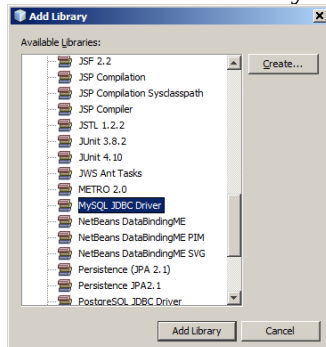
Otra opción:

Si disponemos de un IDE (NetBeans) la forma de registrar el controlador puede diferir ya que aporta por defecto el controlador.



Pero en este caso debemos añadir la librería del conector aparte para evitar problemas de conexión posteriores.

Libraries->Add Library ->



3º.- Ejecución de consultas sobre la base de datos.

Para operar con una base de datos ejecutando las consultas necesarias, nuestra aplicación deberá hacer las operaciones siguientes:

- **Cargar el conector** necesario para comprender el protocolo que usa la base de datos en cuestión.
- **Establecer una conexión** con la base de datos.
- **Enviar consultas SQL** y procesar el resultado.
- **Liberar los recursos** al terminar.
- **Gestionar los errores** que se puedan producir.

Podemos utilizar los siguientes tipos de sentencias:

- **Statement**: para sentencias sencillas en SQL.
- **PreparedStatement**: para consultas preparadas, como por ejemplo las que tienen parámetros.
- **CallableStatement**: para ejecutar procedimientos almacenados en la base de datos.

El API JDBC distingue dos tipos de consultas:

- **Consultas**: SELECT. Para las sentencias de consulta que obtienen datos de la base de datos, se emplea el método ***ResultSet executeQuery(String sql)***. El método de ejecución del comando SQL devuelve un objeto de tipo **ResultSet** que sirve para contener el resultado del comando SELECT, y que nos permitirá su procesamiento.
- **Actualizaciones**: INSERT, UPDATE, DELETE, sentencias DDL. Para estas sentencias se utiliza el método ***int executeUpdate(String sql)***.

4º.- Recuperación de información.

Las consultas a la base de datos se realizan con sentencias SQL que van "**embebidas**" en otras sentencias especiales que son propias de Java. Por tanto, podemos decir que las consultas SQL las escribimos como parámetros de algunos métodos Java que reciben el **String** con el texto de la consulta SQL.

Las consultas devuelven un **ResultSet**, que es una clase java parecida a una lista en la que se aloja el resultado de la consulta. Cada elemento de la lista es uno de los registros de la base de datos que cumple con los requisitos de la consulta.

El objeto **ResultSet** contiene la tabla resultado de la pregunta SQL que se haya realizado.

Ejemplo :

resultado →

nif	nombre	Apellidos	Teléfono
11.111.111	Carlos	Escobar Pérez	654897654
22.222.222	Elena	Pulido López	645321200
33.333.333	Gloria	Fiz García	664231209
44.444.444	Alicia	Sánchez Font	675443312

Con el **ResultSet** hay disponibles una serie de métodos que permiten movernos hacia delante y hacia atrás en las filas, y obtener la información de cada fila.

Cuando se obtiene la tabla resultado, no podemos leer directamente de la misma, necesitamos un **next()** para situarnos en el primer elemento válido.

Por ejemplo, para obtener: nif, nombre, apellidos y teléfono de los clientes que están almacenados en la tabla del mismo nombre, haríamos la siguiente consulta:

// Preparamos la consulta y la ejecutamos

```
Statement s = n.createStatement();
```

```
ResultSet rs = s.executeQuery
```

```
("SELECT NIF, NOMBRE,APELLIDOS, TELÉFONO FROM CLIENTE");
```

El método **next()** del **ResultSet** hace que dicho puntero avance al siguiente registro. Si lo consigue, el método **next()** devuelve **true**. Si no lo consigue, porque no haya más registros que leer, entonces devuelve **false**.

El método **executeQuery** devuelve un objeto **ResultSet** para poder recorrer el resultado de la consulta utilizando un cursor.

Para obtener una columna del registro utilizamos los métodos **get**. Hay un método **get...** para cada tipo básico Java y para las cadenas.

Un método interesante es **wasNull** que nos informa si el último valor leído con un método **get** es nulo.

Cuando trabajamos con el **ResultSet**, en cada registro, los métodos **getInt()**, **getString()**, **getDate()**, etc., nos devuelve los valores de los campos de dicho registro. Podemos pasar a estos métodos un índice (que comienza en 1) para indicar qué columna de la tabla de base de datos deseamos, o bien, podemos usar un **String** con el nombre de la columna (tal cual está en la tabla de base de datos).

Código consulta e iteración.

// Obtener la conexión

```
Connection con = DriverManager.getConnection(connectionUrl);
```

// Preparamos la consulta

```
Statement s = con.createStatement();
```

```
ResultSet rs = s.executeQuery
```

```
("SELECT NIF, NOMBRE,APELLIDOS, TELÉFONO FROM CLIENTE");
```

// Iteramos sobre los registros del resultado

```
while (rs.next())
```

```
    System.out.println (rs.getString("NIF") + " " +
```

```
        rs.getString (2) + " " +
```

```
        rs.getString (3) + " " +
```

```
        rs.getString (4));
```

5º.- Actualización de información.

Respecto a las consultas de actualización, **executeUpdate**, retornan el número de registros insertados, registros actualizados o eliminados, dependiendo del tipo de consulta que se trate.

Supongamos que tenemos varios registros en una tabla Cliente, de una base de datos notarbdb. Si quisiéramos actualizar el teléfono del tercer registro, que tiene idCLIENTE=3 y ponerle como nuevo teléfono el 968610009 tendríamos que hacer:

```
String connectionUrl = "jdbc:mysql://localhost/notarbd" ;
```

// Obtener la conexión

```
Connection con = DriverManager.getConnection(connectionUrl,"root","root");
```

// Preparamos la consulta y la ejecutamos

```
Statement s = con.createStatement();
```

```
s.executeUpdate("UPDATE CLIENTE SET teléfono='968610009' WHERE idCLIENTE=3");
```

// Cerramos la conexión a la base de datos.

```
con.close();
```

6º.- Añadir información.

Si queremos añadir un registro a la tabla Cliente, de la base de datos con la que estamos trabajando tendremos que utilizar la sentencia **INSERT INTO** de SQL. Al igual que hemos visto en el apartado anterior, utilizaremos **executeUpdate** pasándole como parámetro la consulta, de inserción en este caso. Así, un ejemplo sería:

// Preparamos la consulta y la ejecutamos

```
Statement s = con.createStatement();
s.executeUpdate( "INSERT INTO CLIENTE" +
" (idCLIENTE, NIF, NOMBRE, APELLIDOS, DIRECCIÓN, CPOSTAL, TELÉFONO,
CORREOELEC)" + " VALUES (4, '66778998T', 'Alfredo', 'Gates Gates', 'C/ Pirata 23','20400',
'891222112', 'prueba@eresmas.es' )" );
```

Nota : Para insertar el contenido de una variable de tipo VARCHAR se debe introducir entre comillas simples ' '.

7º.- Borrado de información.

Cuando nos interese eliminar registros de una tabla de una base de datos, emplearemos la sentencia SQL: **DELETE**. Así, por ejemplo, si queremos eliminar el registro a la tabla Cliente, de nuestra base de datos y correspondiente a la persona que tiene el nif: 66778998T, tendremos que utilizar el código siguiente.

// Preparamos la consulta y la ejecutamos

```
Statement s = con.createStatement();
numReg = res.executeUpdate( "DELETE FROM CLIENTE WHERE NIF= '66778998T' " );
```

// Informamos del número de registros borrados

```
System.out.println ( "\nSe borró " + numReg + " registro\n" );
```

8º.- Cierre de conexiones.

Las conexiones a una base de datos consumen muchos recursos en el sistema gestor y por tanto en el sistema informático en general. Por ello, conviene cerrarlas con el método **close()** siempre que vayan a dejar de ser utilizadas, en lugar de esperar a que el ‘garbage collector’ de Java las elimine.

También conviene cerrar las consultas (**Statement** y **PreparedStatement**) y los resultados (**ResultSet**) para liberar los recursos.

9º.- Excepciones en JDBC.

En todas las aplicaciones en general, y por tanto en las que acceden a bases de datos en particular, nos puede ocurrir con frecuencia que la aplicación no funciona, no muestra los datos de la base de datos que deseábamos, etc.

Es importante capturar las excepciones que puedan ocurrir para que el programa no aborte de manera abrupta. Además, es conveniente tratarlas para que nos den información sobre si el problema es que se está intentando acceder a una base de datos que no existe, o que el servicio MySQL no está arrancado, o que se ha intentado hacer alguna operación no permitida sobre la base de datos, como acceder con un usuario y contraseña no registrados, ...

Por tanto es conveniente emplear el método **getMessage()** de la clase **SQLException** para recoger y mostrar el mensaje de error que ha generado MySQL, lo que seguramente nos proporcionará una información más ajustada sobre lo que está fallando.

Cuando se produce un error se lanza una excepción del tipo **java.sql.SQLException**.

- Es importante que **las operaciones de acceso a base de datos estén dentro de un bloque try-catch** que gestione las excepciones.
- Como ya se ha apuntado, los objetos del tipo `SQLException` tienen dos métodos muy útiles para obtener el código del error producido y el mensaje descriptivo del mismo, `getErrorCode()` y `getMessage()` respectivamente.

El método **getMessage()** imprime el mensaje de error asociado a la excepción que se ha producido, que aunque esté en inglés, nos ayuda a saber qué ha generado el error que causó la excepción. El método **getErrorCode()**, devuelve un número entero que representa el código de error asociado. Habrá que consultar en la documentación para averiguar su significado.

10º.- Sentencias preparadas para evitar SQLInjection.

- Uso de sentencias preparadas en inserción de datos:

```
public void insertarVotante(Votante _ObjVotante, ..... ) {
    try{
        //Sentencias preparadas
        String ordenSQL="INSERT INTO VOTANTE VALUES(null,?,?,?,?,?,N,'Votante)";
        PreparedStatement PrepStm=_Conexion.prepareStatement(ordenSQL);
        PrepStm.setString(1,_ObjVotante.getNif());
        PrepStm.setString(2,_ObjVotante.getNombre());
        PrepStm.setString(3,_ObjVotante.getApellidos());
        PrepStm.setString(4,_ObjVotante.getDomicilio());
        PrepStm.setDate(5,java.sql.Date.valueOf(_ObjVotante.getFechaNac()));
        PrepStm.setString(6,_ObjVotante.getPassword());
        int filas=PrepStm.executeUpdate();
        //Comprobar si se ha insertado
        if(filas!=1){
            .....
        }
    } catch(SQLException SQLE){
        String MensajeError=SQLE.getMessage();
        int CodigoError=SQLE.getErrorCode();
    }

}

} //insertarVotante
```

Para que se produzca un ataque en una inserción, el atacante debe conocer al menos parte de la estructura de nuestras tablas.

Ejemplo de inserción de código malicioso en un formulario de consulta o modificación (SQLInjection):

En una operación de lectura o actualización en donde utilicemos una clausula WHERE nuestra base de datos se vuelve más vulnerable.

Por ejemplo, si en un campo de texto (type="text") permitimos cualquier cadena de entrada (lo habitual) y no controlamos la longitud, podremos encontrarnos con el siguiente intento en un formulario que pide nif y password. Como valor de nif colocamos uno válido pero acompañado de algo más: **99989677X' or '1'='1'**, es lógico que vaya aplicado a un SELECT con WHERE asociado al nif, por tanto, la comparación podría ser algo así: `WHERE nif='99989677X' or '1'='1' AND password="....."`.

Así pues, la consulta siempre será cierta porque 1 es 1 y está relacionado por OR, y aunque insertemos un password erróneo siempre devolverá los datos de la persona cuyo NIF indicamos.

El consecuente error sería aplicar UPDATE O DELETE creyendo que solo afecta a esa fila. Pero la operación anterior también ira condicionada por el WHERE del nif, o sea, que siempre será cierta para todas las filas de la tabla.

Conclusión, con saber el NIF de las personas registradas sabremos todo sus datos. Incluso podremos modificarlos o borrarlos.

Nota: En el formulario, los controles del NIF con HTML5 o Javascript, pueden hacer imposible el SQLInjection, pero no ocurre así en campos de búsqueda por apellidos, nombre o domicilio.

5.- Tipos SQL en Java

Muchos de los tipos de datos estándar de SQL '92, no tienen un equivalente nativo en Java. Para superar esta deficiencia, se deben mapear los tipos de datos SQL en Java, utilizando las clases JDBC para acceder a los tipos de datos SQL. Es necesario saber cómo recuperar adecuadamente tipos de datos Java; como *int*, *long*, o *String*, a partir de sus contrapartidas SQL almacenadas en base de datos. Esto puede ser especialmente importante si se está trabajando con datos numéricos, que necesiten control decimal con precisión, o con fechas SQL, que tienen un formato muy bien definido.

El mapeo de los tipos de datos Java a SQL es realmente sencillo, tal como se muestra en la tabla que acompaña a este párrafo. Los tipos que comienzan por "java" no son tipos básicos, sino clases que tienen métodos para trasladar los datos a formatos utilizables, y son necesarias estas clases porque no hay un tipo de datos básico que mapee directamente su contrapartida SQL. La creación de estas clases debe hacerse siempre que se necesite almacenar un tipo de dato SQL en un programa Java, para poder utilizar directamente el dato desde la base de datos.

Java	SQL
String	VARCHAR
boolean	BIT
byte	TINYINT
short	SMALLINT
int	INTEGER
long	BIGINT
float	REAL
double	DOUBLE
byte[]-byte array: imágenes, sonidos...	VARBINARY (BLOBs)
java.sql.Date	DATE
java.sql.Time	TIME

java.sql.Timestamp	TIMESTAMP
java.math.BigDecimal	NUMERIC

El tipo de dato *byte[]*, es un array de bytes de tamaño variable. Esta estructura de datos guarda datos binarios, que en SQL son VARBINARY y LONG-VARBINARY. Estos tipos se utilizan para almacenar imágenes, ficheros de documentos, y cosas parecidas. Para almacenar y recuperar este tipo de información de la base de datos, se deben utilizarlos métodos para streams que proporciona JDBC: *setBinaryStream()* y *getBinaryStream()*.

La conversión de tipos en el sentido contrario puede no estar tan clara, ya que hay tipos SQL cuya tipo Java correspondiente puede no ser evidente, como VARBINARY, o DECIMAL, etc. La tabla siguiente muestra los tipos Java correspondientes a cada tipo SQL.

SQL	Java
CHAR	String
VARCHAR	String
LONGVARCHAR	String
NUMERIC	java.math.BigDecimal
DECIMAL	java.math.BigDecimal
BIT	boolean
TINYINT	byte
SMALLINT	short
INTEGER	int
BIGINT	long
REAL	float
FLOAT	double
DOUBLE	double
BINARY	byte[]
VARBINARY	byte[]
LONGVARBINARY	byte[]
DATE	java.sql.Date
TIME	java.sql.Time
TIMESTAMP	java.sql.Timestamp

Existe una constante para cada tipo de dato SQL, declarada en **java.sql.Types**; por ejemplo, el tipo al tipo **TIMESTAMP** le corresponde la constante **java.sql.Types.TIMESTAMP**.

Además, JDBC proporciona clases Java nuevas para representar varios tipos de datos SQL: estas son **java.sql.Date**, **java.sql.Time** y **java.sql.Timestamp**.

6.- Aproximación a SQL estándar.

A modo de resumen, puesto que al alumno se le supone familiarizado con estos conceptos, se describen las principales órdenes de manipulación de tablas en lenguaje SQL estándar.

Creación de Tablas

Para crear y definir una tabla de la base de datos se utiliza el siguiente comando:

CREATE TABLE nombre-tabla
(nombre-columna tipo-datos, tamaño [NOT NULL])
Restricciones

Lo primero que se indica en este comando es el nombre que va a tener la nueva tabla, a continuación, se especifican los nombres de las columnas de la tabla y el tipo y tamaño máximo de los datos que van a contener esas columnas

El tamaño máximo del nombre de la tabla depende de cada implementación concreta

Cada columna debe tener un nombre diferente dentro de la tabla

Hay que definir el tipo de datos que van a ir en cada columna, así como su tamaño

Las definiciones de columnas van separadas por comas

El intento de introducir datos de un tipo diferente en una columna será rechazado

Puede definirse que una columna admita valores nulos o que no los admita (**NOT NULL**)

Las restricciones indican cosas tales como valores únicos, claves ajenas, claves primarias, etc.

La definición se guarda en el diccionario o catálogo

Al ejecutarse el comando **CREATE**, la tabla creada tendrá su definición almacenada en el diccionario, pero estará vacía. Habrá que insertar las filas en ella mediante el comando **INSERT** o alguna utilidad de carga inicial de la que disponga el producto concreto que se esté utilizando

Ejemplos:

```
CREATE TABLE empleados
(num_emp NUMBER(4) PRIMARY KEY,
nombre VARCHAR2 (10) NOT NULL,
puesto VARCHAR2 (9),
num-sup NUMBER(4),
fecha-ingreso DATE,
salario NUMBER(7,2),
comisión NUMBER (7,2),
num-dept NUMBER(2);

CREATE TABLE departamentos
(num_dept NUMBER(2) PRIMARY KEY ,
nombre VARCHAR2 (15) NOT NULL UNIQUE,
localidad VARCHAR2 (15));
```

El comando CREATE INDEX de SQL permite crear un índice para una tabla. Su formato es:

***CREATE [UNIQUE] INDEX nombre-índice ON tabla
(columnas y orden-clasificación);***

La tabla referenciada debe existir.

Un índice puede ser creado sobre una columna o sobre varias columnas de la tabla.

El orden en el que se especifiquen las columnas que forman el índice es significativo.

En una misma tabla se pueden crear varios índices.

Ejemplos:

*CREATE INDEX nombre
ON empleados
(nombre);*

*CREATE INDEX dept_puesto
ON empleados
(num_dept, puesto);*

Con el comando ALTER TABLE se pueden añadir nuevas columnas a una tabla, añadir o quitar restricciones, cambiar la definición de columnas y modificar la información sobre el almacenamiento de los datos de la tabla. En algunas implantaciones del producto también se pueden borrar columnas existentes de una tabla

Los cambios se almacenarán en el diccionario de datos

Si la columna que añadimos no admite nulos, al poner NOT NULL nos dará un error. La deberemos de crear normal y después la modificaremos para que sea no nula, pero antes debemos de llenar cada fila de la columna con datos

El formato del comando ALTER TABLE es:

***ALTER TABLE nombre-tabla
modificaciones;***

La tabla indicada por "nombre-tabla" debe existir

"modificaciones" son las especificaciones de los cambios a realizar:

- Añadir o quitar columnas
- Añadir o quitar restricciones
- Cambiar la definición de columnas

Cuando se quiere añadir una nueva columna a una tabla el formato de] comando ALTER es:

***ALTER TABLE nombre-tabla
ADD (nombre-columna definición);***

Ejemplo:

*ALTER TABLE empleados
ADD (titulacion CHAR(15));*

Todas las filas existentes de la tabla contendrán un valor nulo en la nueva columna

El comando DROP TABLE del SQL se utiliza para borrar la definición de una tabla de la base de datos.

Su formato es:

DROP TABLE nombre-tabla;

Algunas implantaciones del producto borran la definición de la tabla y todas las filas que pudiera contener. En otros, sin embargo, es necesario que la tabla esté vacía para poder borrar su definición. Normalmente, cuando se suprime una tabla se suprimen automáticamente todos los índices y vistas definidas sobre esa tabla.

Podemos borrar todas las filas y liberar el espacio ocupado por una tabla, sin eliminar su definición con:

TRUNCATE TABLE nombre_tabla

Ejemplo:

DROP TABLE grados;

El comando DROP INDEX del SQL se utiliza para borrar la definición de un índice. Su formato es:

DROP INDEX nombre-índice;

Ejemplo:

DROP INDEX Ind_uno;

Recuperar Información

El comando SELECT se utiliza para seleccionar datos de una o más tablas, el formato es:

SELECT $\left\{ \begin{array}{l} \text{ALL} \quad [*] \\ \text{DISTINCT} \end{array} \right\}$ *columnas/expresiones*

FROM tablas

[clausula WHERE]

[clausula GROUP BY]

[clausula HAVING]

[clausula ORDER BY];

El comando SELECT básico tiene dos partes, denominadas cláusulas: La cláusula SELECT, que indica los datos que se van a seleccionar y la cláusula FROM, que indica la tabla o tablas que se van a utilizar en la selección

SELECT nombre, puesto, salario
FROM empleados

El resultado de un comando SELECT es siempre una tabla

Cuando se quiere obtener toda la información contenida en una tabla, el formato del comando SELECT es:

SELECT * FROM nombre-tabla;

Ejemplo: Obtener toda la información de la tabla EMPLEADOS

*SELECT * FROM empleados;*

Cuando se quieran obtener todas las filas, pero solo algunas columnas de la tabla, (operación PROYECCIÓN del álgebra relacional), el formato del comando SELECT es:

SELECT nombres-columnas FROM nombre-tabla;

Ejemplo: Seleccionar el numero y nombre de todos los empleados

SELECT num_emp, nombre
FROM empleados;

Para seleccionar determinadas filas de una tabla hay que imponer condiciones que deberán cumplir las filas que se seleccionen

Para ello se utiliza la cláusula WHERE

El formato del comando SELECT con la cláusula WHERE es:

SELECT $\left\{ \begin{array}{c} * \\ \text{columnas} \end{array} \right\}$ ***FROM tablas***

WHERE condición;

donde "condición" indica las características que deben satisfacer las filas elegidas

Ejemplo 1: Sacar nombre, puesto y fecha de ingreso de los empleados del departamento 10

SELECT nombre, puesto, fecha-ingreso
FROM empleados
WHERE num_dept = 10;

Ejemplo 2: Sacar nombre, puesto y salario de los empleados que ganen mas de 2000.

```
SELECT nombre, puesto, salario  
FROM empleados  
WHERE salario > 2000;
```

Almacenar Información

El comando INSERT del SQL permite insertar filas dentro de las tablas.

Uno de sus formatos es:

INSERT INTO tabla [(columnas)] VALUES (valores);

La tabla en la que se van a insertar filas debe estar definida previamente

Los valores deben corresponder en posición con la lista de columnas

Cada valor insertado debe coincidir con el tipo de datos de cada columna correspondiente, o ser un valor nulo (NULL)

Los valores de tipo carácter deben ir entre apóstrofes. Los valores numéricos o NULL no van entre apóstrofes

El comando INSERT se puede utilizar con una cláusula SELECT para insertar en una tabla filas seleccionadas de otra tabla. En este caso, el formato es:

INSERT INTO tabla [(columnas)] cláusula SELECT

Las columnas seleccionadas con la SELECT deberán tener tipos de datos compatibles con los de las columnas de la tabla donde se van a insertar

Ejemplos:

```
INSERT INTO empleados  
VALUES (7950, 'RAMIREZ', 'ADMINISTRATIVO', 7782, '20-oct-83', 1000, NULL, 10);
```

```
INSERT INTO departamentos  
VALUES (50, 'FABRICACIÓN', 'MURCIA');
```

```
INSERT INTO empleados  
(nombre, num_emp, puesto, fecha_ingreso, num_dept)  
VALUES ('VERDI', 8100, 'ANALISTA', '20-apr-83', 20);
```

```
INSERT INTO departamentos  
(nombre, num_dept)  
VALUES ('MARKETING', 60);
```

```
INSERT INTO analisis  
SELECT * FROM empleados WHERE puesto = 'ANALISTA'
```

Borrar Información

El comando DELETE del SQL se utiliza para borrar filas de una tabla. Su formato es:

DELETE FROM tabla [cláusula WHERE];

Si se omite la cláusula WHERE se borran todas las filas de la tabla.

Si se especifica la cláusula WHERE se borran aquellas filas que satisfagan las condiciones impuestas en ella.

Aunque se borren todas las filas de una tabla no desaparece la definición de la misma.

Con un solo comando DELETE se puede borrar una fila, varias filas o todas las filas de una tabla.

Ejemplos:

*DELETE FROM empleados
WHERE salario > 2500 AND puesto = 'ANALISTA';*

DELETE FROM grados ;

*DELETE FROM empleados
WHERE puesto = 'ADMINISTRATIVO'
AND salario > 900
AND num_dept IN
(SELECT num_dept
FROM departamentos
WHERE localidad = 'SEVILLA');*

Actualizar Información

El comando UPDATE del SQL se utiliza para modificar valores en filas existentes de las tablas. Su formato es:

***UPDATE tabla
SET especificación-cambio
[cláusula WHERE] ;***

SET indica qué columnas se van a cambiar y cuáles serán los nuevos valores de esas columnas.

"Especificación-cambio" tiene el formato:

columna = valor, columna = valor,

Si se omite la cláusula WHERE, la modificación afecta a todas las filas de la tabla

Con la cláusula WHERE se puede seleccionar las filas a las que va a afectar el cambio

Con un solo comando UPDATE se puede actualizar una sola fila, varias filas o todas las filas de una tabla

Ejemplos:

```
UPDATE empleados  
SET puesto = 'JEFE',  
    salario = salario * 1.1  
WHERE nombre = 'ARRIBAS';
```

```
UPDATE empleados  
SET    salario = 5000,  
      puesto   'MANAGER'  
WHERE puesto = 'JEFE';
```

Serialización

1.- Serialización

La **serialización** es un proceso por el que un objeto cualquiera se puede convertir en una **secuencia de bytes** con la que más tarde se podrá reconstruir dicho objeto manteniendo el valor de sus variables. Esto permite guardar un objeto en un archivo o mandarlo por la red.

Para que una clase pueda utilizar la serialización, debe implementar la interface **Serializable**, que no define ningún método. Casi todas las clases estándar de **Java** son serializables. La clase **MiClase** se podría serializar declarándola como:

```
public class MiClase implements Serializable { }
```

Para escribir y leer objetos se utilizan las clases **ObjectInputStream** y **ObjectOutputStream**, que cuentan con los métodos **writeObject()** y **readObject()**.

Para grabar un objeto:

La manera de proceder es crear un objeto de la clase **FileOutputStream**, que será el que identifique el fichero donde se va a guardar el objeto serializado.

Después, crear un objeto **ObjectOutputStream** relacionado con el anteriormente creado.

A través de este último objeto, usar el método **writeObject(Object O)** y pasarle como argumento el objeto motivo de la serialización (O).

Para leerlo:

Crear un objeto de la clase **FileInputStream** que identifique el fichero que contiene el objeto antes guardado.

Después crear otro de la clase **ObjectInputStream** y relacionarlo con el anterior.

A través de este último objeto, utilizar el método **readObject()**, teniendo en cuenta que extraerá el objeto almacenado en el fichero con formato **Object**, se precisará un casting para su posterior uso.

Al serializar un objeto, automáticamente se serializan todas sus variables y atributos. A su vez se serializan los objetos correspondientes si los atributos son referencias (todos deben ser serializables).

También se reconstruyen de igual manera.

2.- Ejemplo de Lectura y Escritura de Objetos.

Una manera frecuente de guardar varios objetos del mismo tipo es estructurarlos en un **ArrayList** y grabar y/o leer dicha estructura de forma adecuada.

Ejemplo:

```
public class Carta implements java.io.Serializable{
    private String palo;
    private int numero;
    public Carta(String _palo, int _numero){
        palo=_palo;
        numero=_numero; }
    public String toString(){
        return numero + " "+palo;}
}
```

```
import java.util.ArrayList;
import java.io.*;
public class Escribe_Lee_Cartas {
    public static void main(String[] args) {
        ArrayList Cartas=new ArrayList();
        ArrayList Cartas_Aux=new ArrayList();
        //Crear el ArrayList Cartas //
        Cartas.add(new Carta("oros",3));
        Cartas.add(new Carta("copas",12));
        Cartas.add(new Carta("espadas",1));
        Cartas.add(new Carta("bastos",5));
        Cartas.add(new Carta("oros",7));
        //Guardo el ArrayList en un fichero //
        try{
            FileOutputStream FOS=new FileOutputStream("fichero.dat");
            ObjectOutputStream OOS= new ObjectOutputStream(FOS);
            OOS.writeObject(Cartas);
            OOS.close();
        }
        catch(IOException ioe){
            System.out.println("IOException : "+ ioe);
        }
    }
}
```

```
//Leo el ArrayList guardado //
try{
    FileInputStream FIS=new FileInputStream("fichero.dat");
    ObjectInputStream OIS= new ObjectInputStream(FIS);
    Cartas_Aux=(ArrayList)OIS.readObject();
    OIS.close();
}
catch(IOException ioe){
    System.out.println("IOException : "+ ioe);
}
catch(ClassNotFoundException cnfe){
    System.out.println("ClassNotFoundException : "+ cnfe);
}
//Escribo el contenido del ArrayList //
for(int i=0;i<Cartas_Aux.size();i++){
    System.out.println(Cartas_Aux.get(i));
}

}
}
```